SOMAIYA VIDYAVIHAR UNIVERSITY

Mini-Project topic

SafeShare
A Secure File-Sharing Web Application

Submitted in the partial fulfillment of the

requirements for the course of

Honours Program in Cyber Security and Forensics

by

Harshit Rawat
Roll No. 16024024059

Kaushiki Choubey
Roll No. 16014024013

Avaneesh Nair
Roll No. 16014024046

Of Dept. Computer Engineering

Index

Sr. No.	Contents	Page No.
	Abstract	3
1.	Introduction	3
2.	Objectives and Problem Statement	4
3.	Tools and Technologies Used	4
4.	Methodology and Implementation	5
5.	Results and Screenshots	7
6.	Conclusion and Future Scope	10
7.	References (AI Tools Prompts)	11
8.	Appendix (Source Code)	12

Abstract

SafeShare is a web-based secure file-sharing application developed to demonstrate the practical application of cybersecurity concepts in a file-management workflow. It enables registered users to upload, store, download, and share files while applying authentication, authorization, and encryption controls. The application uses Python and Flask for backend processing, bcrypt for password hashing, JWT-based authentication, and Fernet for file encryption. Neon PostgreSQL stores structured application records, while Supabase Storage holds encrypted file objects in a private bucket. The application is deployed on Render, with source code maintained in GitHub. Additional functionality includes sharing-permission management, administrative user controls, and a password-reset workflow designed around time-limited tokens. The project demonstrates how security controls work together, while recognizing that end-to-end validation, secure key management, and further security testing remain important for a production-grade system.

1. Introduction

1.1 Brief Overview of the Project

SafeShare is a web-based application designed to let registered users upload, store, download, and share files through a controlled workflow. The project combines a browser interface with a server-side application that handles authentication, file operations, and access permissions. It is developed as a mini-project for the Honours Program in Cyber Security and Forensics, with emphasis on applying security concepts to a practical software system.

The application uses Python and Flask for backend request handling. Neon PostgreSQL stores structured production data, while Supabase Storage stores file objects in a private bucket. The application is deployed on Render and its source code is maintained in GitHub.

1.2 Purpose and Motivation

The motivation for SafeShare is to demonstrate that file transfer should be supported by controls for identity, confidentiality, and authorization. If files are stored publicly or the backend does not validate permissions, sensitive content may be exposed. SafeShare routes protected file operations through the application and includes user and administrator workflows.

1.3 Technologies Used

1. Programming language: Python.
2. Backend framework: Flask.
3. Frontend: HTML, CSS, JavaScript, and server-rendered templates.
4. Password security: bcrypt password hashing.
5. Authentication: JWT-based authentication using HTTP-only cookies.
6. File protection: Fernet authenticated symmetric encryption through the Python cryptography library.
7. Relational database: Neon PostgreSQL for production records.
8. Object storage: Supabase Storage with a private bucket.
9. Deployment and source control: Render, Git, and GitHub.

2. Objectives and Problem Statement

2.1 Problem Statement

File-sharing systems must ensure that only authenticated and authorized users can access protected files. A public or guessable link is not a sufficient security boundary, because a client may call backend endpoints directly. A suitable system should validate identity and permissions on the server, protect stored file contents, and allow file owners to manage sharing permissions.

2.2 Objectives

1. Develop a browser-based application for user registration, login, and file management.
2. Hash passwords rather than storing them as plaintext.
3. Encrypt file contents before storage and decrypt them as part of an authorized download workflow.
4. Separate relational application records from file-object storage.
5. Allow users to share files with other registered users and revoke sharing permissions.
6. Implement role-based access control for ordinary users and administrators.
7. Include safeguards for sensitive administrator actions, including last-administrator protections.
8. Provide a password-reset workflow based on time-limited tokens.
9. Deploy the application and verify the main user workflows.

3. Tools and Technologies Used

Technology	**Purpose in SafeShare**
Python	Backend programming language.
Flask	Handles HTTP requests, application routes, and responses.
HTML, CSS, JavaScript	Builds and styles the browser interface.
bcrypt	Hashes passwords and verifies submitted passwords.
JWT	Represents authentication claims used by the application.
Fernet / cryptography	Encrypts and decrypts file content and authenticates encrypted tokens.
Neon PostgreSQL	Stores users, file metadata, sharing relationships, and reset-token records.
Supabase Storage	Stores encrypted file objects in a private bucket.
Render	Hosts the Flask web application.
Git and GitHub	Version control and source-code hosting.

3.1 Environment and Secret Management

Deployment secrets, including database credentials, authentication secrets, and file-encryption keys, are supplied through environment variables. They should not be committed to the

repository or included in screenshots and reports. The file-encryption key must remain available to decrypt existing stored files.

3.2 Security Design Rationale

Password hashing and file encryption solve different problems. bcrypt is used for password storage, while Fernet is used for reversible file-content protection. JWT-based authentication supports identity checks, and server-side authorization determines whether a user can perform a particular action.

4. Methodology and Implementation

4.1 System Architecture

Component	Responsibility
User / Browser	Submits login, upload, download, sharing, and administration requests.
Flask application on Render	Validates input, authenticates the user, checks permissions, and coordinates operations.
Neon PostgreSQL	Stores user records, file metadata, sharing relationships, and reset-token records.
Supabase Storage	Stores encrypted file objects in a private storage bucket.
GitHub repository	Maintains source code and provides the source connection for deployment.

Request flow: Browser → Flask application → authentication and authorization checks → database and/or private object storage → response to the browser.

4.2 Main Modules

Module	Responsibility
app.py	Defines routes and coordinates user, file, sharing, and administration workflows.
auth_utils.py	Provides authentication-related helper functions.
crypto_utils.py	Provides file encryption and decryption operations.
db.py	Initializes and accesses structured application records.
config.py	Loads application configuration and environment settings.
email_utils.py	Supports reset-message delivery when SMTP is configured.
templates/ and static/	Provide HTML templates, styles, JavaScript, and interface assets.

4.3 Registration and Login

Users register and sign in through browser forms. Passwords are processed using bcrypt rather than stored as plaintext. Following successful authentication, the application establishes its JWT-based authentication context using an HTTP-only cookie. Protected routes must validate the authentication context and apply authorization checks before sensitive operations.

4.4 File Upload and Encryption

1. The user selects a file and submits it through the browser.
2. The Flask backend validates the request according to configured rules and limits.
3. The file bytes are processed through the Fernet encryption workflow.
4. The encrypted object is stored in private Supabase Storage.
5. Related file metadata is stored in PostgreSQL.

The current workflow processes file content in memory during encryption and upload. Memory availability and hosting request limits can affect practical maximum file size. Streaming or chunked processing is a possible improvement.

4.5 File Download, Sharing, and Revocation

For a download, the backend checks the user's authentication and access rights before retrieving the encrypted object. The object is decrypted and returned to an authorized user. File owners can grant access to registered users and revoke a sharing permission. Revocation should prevent future access through that grant, but cannot delete a copy that the recipient has already downloaded.

4.6 Administrative Functions

The administrative dashboard supports user-management and role-management operations. Safeguards are included to prevent self-role changes through the role-management action and to prevent demotion or removal of the last administrator. These operations should be demonstrated with test accounts.

4.7 Password Reset

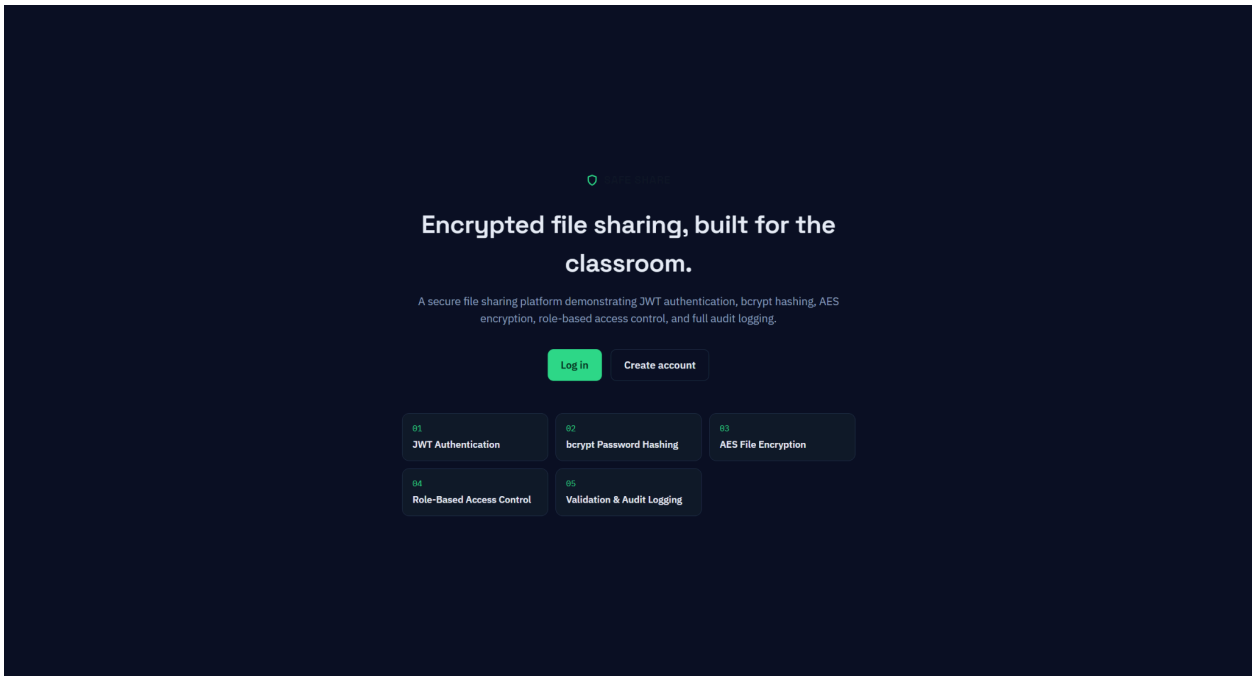
The password-reset workflow uses a generated token whose hash is stored in the database. Intended controls include token expiry after 15 minutes, single-use validation, a generic request response to reduce account enumeration, and request-rate limiting. Email delivery requires valid SMTP configuration; the full email path should only be described as operational after an end-to-end test.

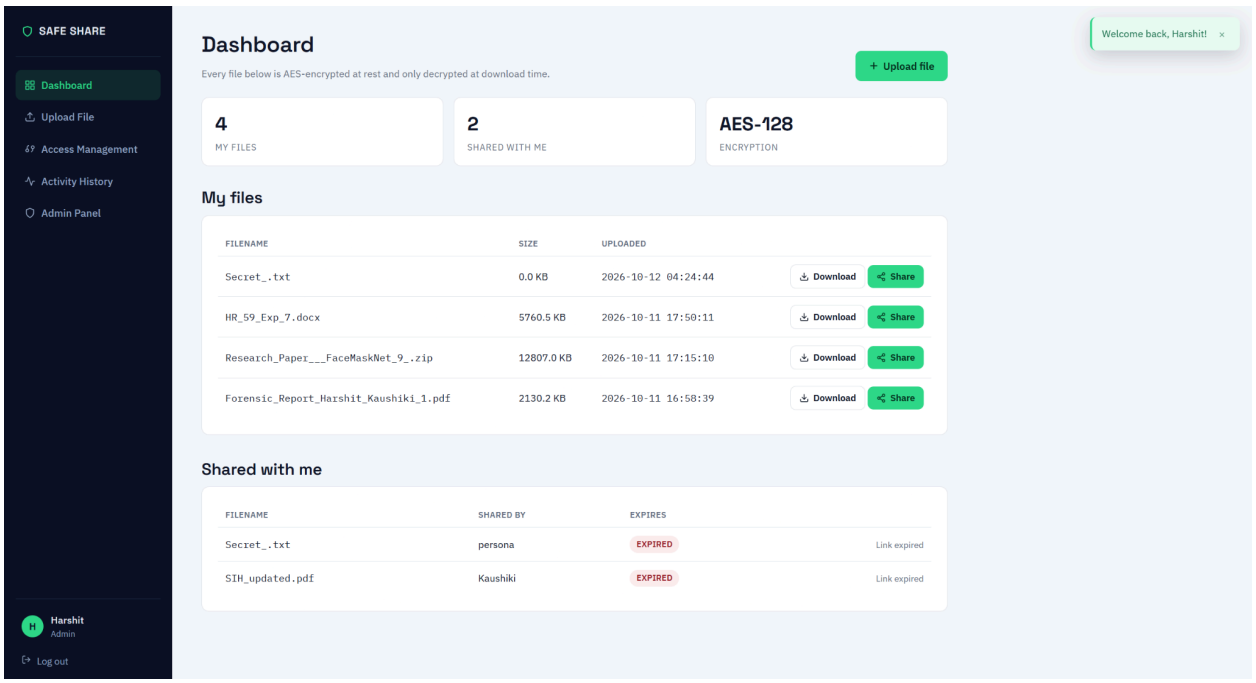
4.8 Deployment

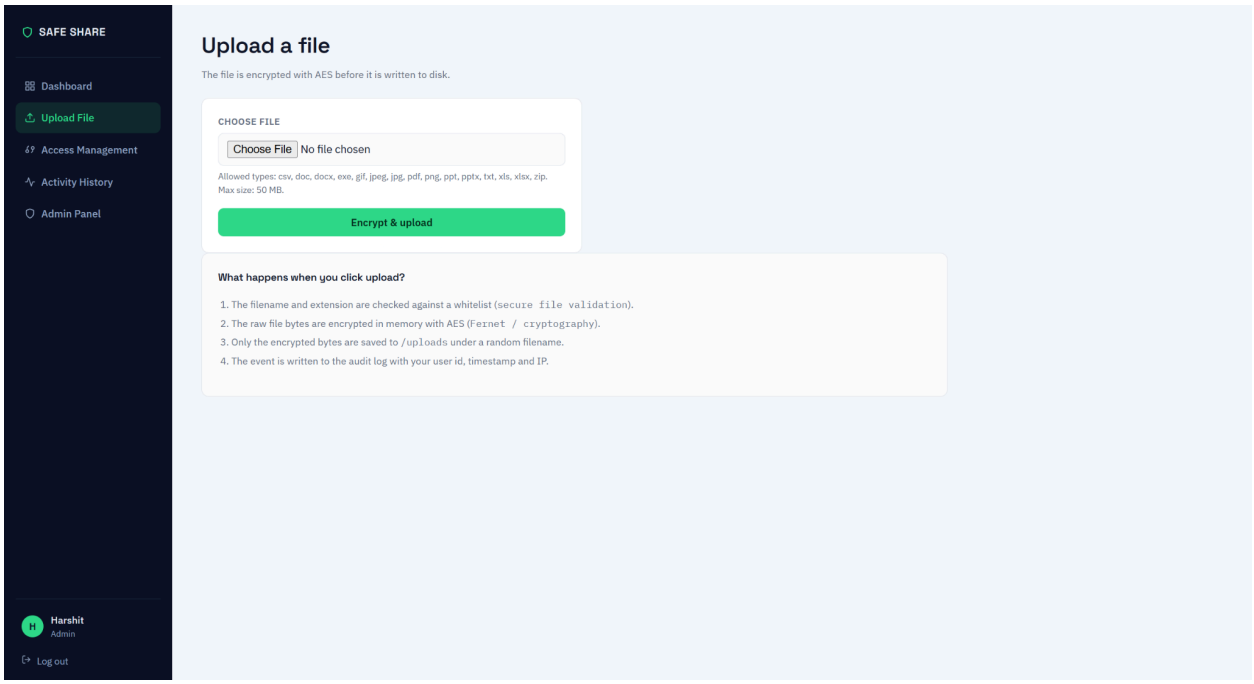
The application is hosted on Render. GitHub maintains the source repository, Neon PostgreSQL stores structured production data, and Supabase Storage stores encrypted file objects. Configuration and secrets are supplied through environment variables and must not be committed to the source repository.

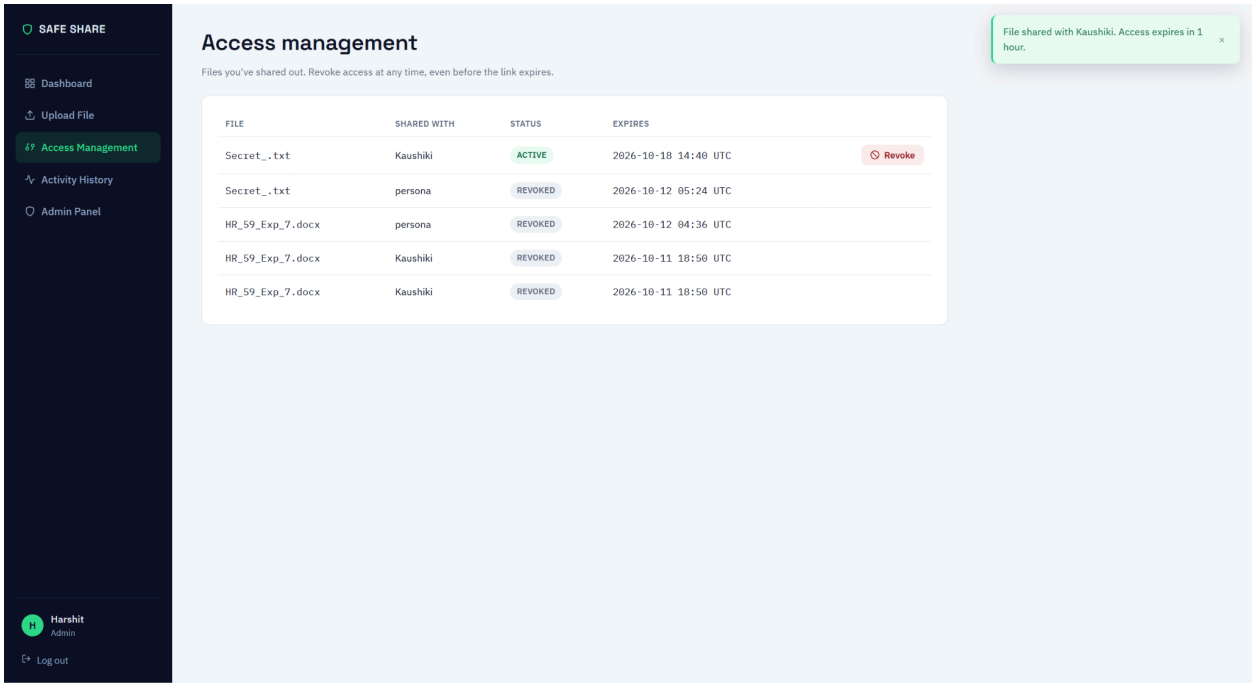
5. Results and Screenshots

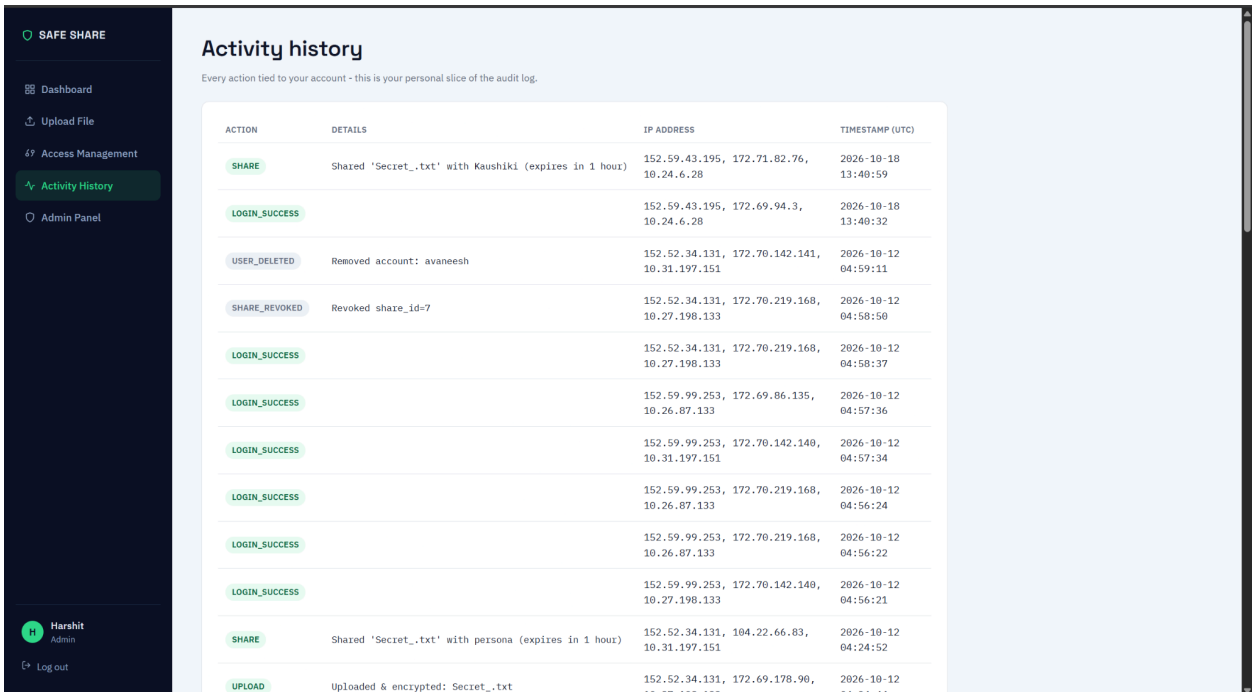
The deployed application is available at https://safeshare-ftpm.onrender.com/. The system interface supports registration and login, a file-management dashboard, upload and download workflows, sharing controls, administrative user management, and a password-reset request page. The following results describe the intended user-facing functions; individual workflows should be demonstrated using test accounts during evaluation.

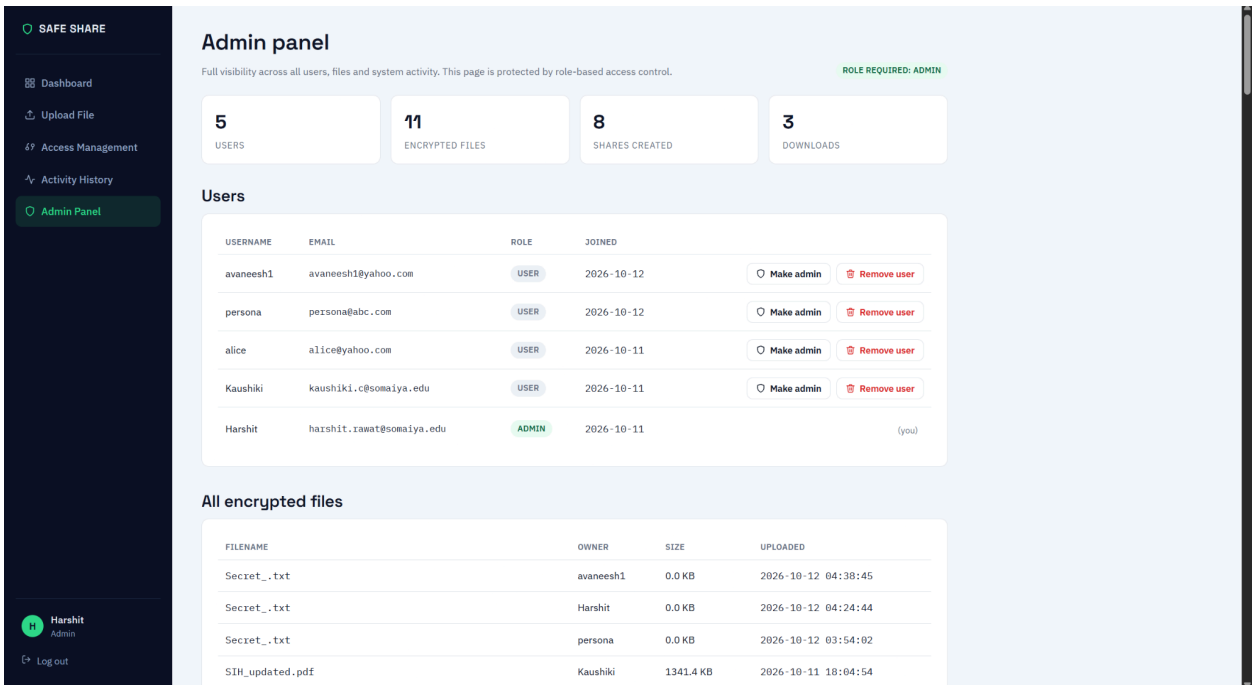





SAFE SHARE
Dashboard
Upload File
Access Management
Activity History
Admin Panel
Harshit
Admin
Log out
Upload a file
The file is encrypted with AES before it is written to disk.
CHOOSE FILE
Choose File No file chosen
Allowed types: csv, doc, docx, exe, gif, jpeg, jpg, pdf, png, ppt, pptx, txt, xls, xlsx, zip.
Max size: 50 MB.
Encrypt & upload
What happens when you click upload?
1. The filename and extension are checked against a whitelist (secure file validation).
2. The raw file bytes are encrypted in memory with AES (Fernet / cryptography).
3. Only the encrypted bytes are saved to /uploads under a random filename.
4. The event is written to the audit log with your user id, timestamp and IP.


SAFE SHARE
Dashboard
Upload File
Access Management
Activity History
Admin Panel
Harshit
Admin
Log out
Access management
Files you've shared out. Revoke access at any time, even before the link expires.
File shared with Kaushiki. Access expires in 1 hour.
FILE
SHARED WITH
STATUS
EXPIRES
Secret_.txt
Kaushiki
ACTIVE
2026-10-18 14:40 UTC
Revoke
Secret_.txt
persona
REVOKED
2026-10-12 05:24 UTC
HR_59_Exp_7.docx
persona
REVOKED
2026-10-12 04:36 UTC
HR_59_Exp_7.docx
Kaushiki
REVOKED
2026-10-11 18:50 UTC
HR_59_Exp_7.docx
Kaushiki
REVOKED
2026-10-11 18:50 UTC





5.1 Application Output and Demonstrated Functions

Function	Application output	Purpose
Login and registration	Browser-based account access	Account creation and authentication.
Dashboard	User file-management interface	Lists available file operations and user files.
File upload/download	File transfer workflow	Encrypted storage and authorized retrieval.

Sharing and revocation	File-sharing controls	Grant or revoke access for registered users.
Admin dashboard	Administrative interface	Role and user-management controls.
Password reset	Password-reset request page	Starts the reset workflow; delivery depends on SMTP configuration.

5.2 Functional Checks

Test	Purpose	Status / note
Application startup	Confirm Flask application imports and initializes routes.	Successful in local checks.
Login and registration pages	Confirm primary authentication pages render.	Loaded in local browser checks.
Dashboard and protected pages	Confirm main pages are reachable in an authenticated workflow.	Loaded in local browser checks.
Upload and download	Verify a file round-trip.	Repeat on the current deployed version before claiming success.
Sharing and revocation	Verify recipient access and denial after revocation.	Run the full two-account workflow on the deployed version.
Administrative action	Check user-management behavior in a test setup.	A local test action returned a redirect and an admin page loaded.
Password reset	Open reset request page and test email delivery.	Page loaded; email delivery depends on SMTP configuration.
Cloud deployment	Inspect Render deployment status.	Commit 64eba32 was shown as Live in the Render dashboard.

5.3 Performance and Reliability

The cloud deployment allows browser access without local installation. The free hosting tier may spin down after inactivity and introduce a cold-start delay. Large uploads may be constrained by in-memory processing and hosting resources. The listed checks are functional checks, not a formal performance benchmark or independent security audit.

6. Conclusion and Future Scope

6.1 Conclusion

SafeShare demonstrates the integration of secure web application concepts into a file-sharing workflow. It combines bcrypt password hashing, JWT-based authentication, Fernet file encryption, server-side authorization, private object storage, relational metadata, sharing-permission management, and administrative safeguards. The project provided practical

experience in secure web development, cryptographic API usage, identity and access management, database integration, and cloud deployment. It should be treated as an educational prototype that requires further validation and hardening before production use.

6.2 Advantages

1. Provides a single interface for file upload, download, and controlled sharing.
2. Uses password hashing rather than storing plaintext passwords.
3. Uses a Fernet-based encryption workflow for file content.
4. Separates structured records from stored file objects.
5. Supports permission revocation and role-based administrative functions.
6. Is deployed to a cloud-hosted environment for browser-based access.

6.3 Limitations

1. The free hosting tier may introduce a cold-start delay after inactivity.
2. File encryption and upload currently use in-memory processing, limiting practical large-file handling.
3. Password-reset email delivery depends on SMTP configuration and end-to-end validation.
4. Security depends on protection of the encryption key, database credentials, storage service key, and other secrets.
5. Revocation cannot remove a file copy that a recipient has already downloaded.
6. The project has not been claimed to have passed an independent penetration test or formal certification.

6.4 Future Scope

1. Implement streaming or chunked file encryption and upload to reduce memory use.
2. Add automated tests for authentication, ownership checks, sharing, revocation, and administrator safeguards.
3. Conduct a security review covering IDOR/BOLA, CSRF, XSS, injection, session handling, and rate-limit bypass.
4. Complete and verify SMTP-based password-reset delivery.
5. Add carefully designed audit logs without logging secrets or file contents.
6. Document encryption-key rotation, backup, recovery, and incident-response procedures.
7. Add monitoring, health checks, and alerting.

7. References and AI Tools Prompts

1. Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/
2. Python Cryptographic Authority. Cryptography Documentation — Fernet. https://cryptography.io/en/latest/fernet/
3. bcrypt package documentation. https://pypi.org/project/bcrypt/
4. OWASP Foundation. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/
5. OWASP Foundation. Authentication Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

6. OWASP Foundation. Authorization Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
7. Render Documentation. Web Services. https://render.com/docs/web-services
8. Supabase Documentation. Storage. https://supabase.com/docs/guides/storage
9. Neon Documentation. https://neon.tech/docs
10. Jones, M., Bradley, J., and Sakimura, N. JSON Web Token (JWT), RFC 7519. https://www.rfc-editor.org/rfc/rfc7519

7.1 AI Tools Prompts

AI tools were used as supporting aids during project development and report preparation. ChatGPT was used to help explain implementation concepts, organize report content, and clarify cybersecurity topics such as authentication, file encryption, and access control. Claude was used as an additional AI assistant for reviewing and refining explanations and written material. The tools supported the team's work; the application design, configuration, testing, and final verification remain the team's responsibility.

8. Appendix (Source Code)

The project source code is maintained in the GitHub repository: https://github.com/ROGHET/SafeShare. The main implementation files include app.py, auth_utils.py, crypto_utils.py, db.py, config.py, and email_utils.py, together with HTML templates and static assets. The repository and any code submitted separately must exclude environment files, credentials, access tokens, database URLs, and encryption keys.

8.1 Demonstration Workflow

1. Open the deployed SafeShare website and wait for the service to respond.
2. Sign in using a prepared test account.
3. Upload a small, non-sensitive sample file.
4. Download the file and confirm that it opens correctly.
5. Share the file with a second test account and verify access.
6. Revoke the share and confirm that access through that permission is denied.
7. Open the administrator dashboard and explain the role safeguards.
8. Show the password-reset page and state that email delivery depends on SMTP configuration.
9. Show the deployment status and summarize the architecture.

8.2 Team Members

Name	Roll number
Harshit Rawat	16024024059
Kaushiki Choubey	16014024013
Avaneesh Nair	16014024046